Algorithm Project

Triangle Project

Team members:

- صالح أسامة صالح	20240493
- احمد طاهر عبد العظيم	20240047
- احمد محمد سمير	20240078
- شادي هاني عبد المنعم	20240474
- مصطفى وليد سيد	20240979
- عبد الله عماد كارم	20240577

First Algorithm (Non-Recursive):

Idea:

The algorithm sorts the array using Bubble Sort, then checks every 3 consecutive numbers.

If the sum of the first two numbers is greater than the third number,

then a triangle exists.

Pseudo-code:

BubbleSort(A)

1. For i from 0 to length(A)-1
2. For j from 0 to length(A)-i-2
3. If A[j] > A[j+1]
4. Swap A[j] and A[j+1]

TriangleNonRecursive(A)

1. BubbleSort(A)
2. For i from 0 to length(A)-3
3. If A[i] + A[i+1] > A[i+2]
4. Return 1
5. Return 0

Display on Screen

1. Display Enter Inputs
2. For Loop to read inputs
3. Call (triangleNonRecursive)
4. Display output

Implementation:

```
#include <stdio.h>
/* Bubble Sort */
void sortArray(int A[], int N)
{
   int i, j, temp;
   for(i = 0; i < N - 1; i++)
   {
      for(j = 0; j < N - i - 1; j++)
      {
         if(A[j] > A[j + 1])
         {
            temp = A[j];
            A[j] = A[j + 1];
            A[j + 1] = temp;
         }
      }
   }
}
/* Triangle Check */
int triangleNonRecursive(int A[], int N)
{
   int i;
   /* Sort the array */
   sortArray(A, N);
   /* Check triangle condition */
   for(i = 0; i < N - 2; i++)
```

```
    {
        if(A[i] + A[i + 1] > A[i + 2])
        {
            return 1;
        }
    }
    return 0;
}
int main()
{
    int N;
    int i;
    printf("Non-Recursive Algorithm to Check is there a Triangle \n");
    printf("\nEnter number of Inputs: ");
    scanf("%d", &N);
    int A[N];
    /* Input array elements */
    for(i = 0; i < N; i++)
    {
        printf("Enter side %d: ", i + 1);
        scanf("%d", &A[i]);
    }
    int result;
    result = triangleNonRecursive(A, N);
    if(result == 1)
    {
        printf("\n  {1}  Triangle exists\n");
```

```
    }
    else
    {
        printf("\n  {0}  Triangle does not exist\n");
    }
    return 0;
}
```

Analysis & complexity (steps):

Sorting Complexity: $O(n^2)$

Traversal Complexity: $O(n)$

Final Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Second Algorithm (Recursive):

Idea:

The algorithm sorts the array, then uses recursion to check every triplet.

Pseudo-code:

Merge(A, left, mid, right)

1. Create two temporary arrays:

    L[] for left half
    R[] for right half

2. Copy elements into L[] and R[]

3. Compare elements from L[] and R[]
    Place smaller element into original array A[]

4. Copy remaining elements from L[]

5. Copy remaining elements from R[]

MergeSort(A, left, right)

1. If left < right

2. mid = (left + right) / 2

3. MergeSort(A, left, mid)

4. MergeSort(A, mid + 1, right)

5. Merge(A, left, mid, right)

CheckTriangle(A, N, i)

1. If i >= N - 2
    Return 0

2. If A[i] + A[i+1] > A[i+2]

    Return 1

3. Return CheckTriangle(A, N, i + 1)

TriangleRecursive(A, N)

1. MergeSort(A, 0, N - 1)

2. Return CheckTriangle(A, N, 0)

Display on Screen

1. Display Enter Inputs
2. For Loop to read inputs
3. Call (triangleRecursive)
4. Display output

Implementation:

```
#include <stdio.h>


void merge(int A[], int left, int mid, int right)
{
   int i, j, k;

   int n1 = mid - left + 1;
   int n2 = right - mid;

   int L[n1], R[n2];

   for(i = 0; i < n1; i++)
        L[i] = A[left + i];

   for(j = 0; j < n2; j++)
       R[j] = A[mid + 1 + j];

   i = 0;
   j = 0;
   k = left;

   while(i < n1 && j < n2)
   {
          if(L[i] <= R[j])
          {
```

```
                A[k] = L[i];
                i++;
        }
        else
        {
                A[k] = R[j];
                j++;
        }

        k++;
    }

    while(i < n1)
    {
          A[k] = L[i];
          i++;
          k++;
    }

    while(j < n2)
    {
        A[k] = R[j];
        j++;
        k++;
     }
}

/* Recursive Merge Sort */
void mergeSort(int A[], int left, int right)
 {
       if(left < right)
       {
          int mid = (left + right) / 2;

          mergeSort(A, left, mid);

          mergeSort(A, mid + 1, right);

         merge(A, left, mid, right);
    }
}

/* Recursive Triangle Check */
int checkTriangle(int A[], int N, int i)
```

```
{
    if(i >= N - 2)
    {
        return 0;
    }

    if(A[i] + A[i + 1] > A[i + 2])
    {
            return 1;
    }

    return checkTriangle(A, N, i + 1);
}

   /* Fully Recursive Triangle Algorithm */
   int triangleRecursive(int A[], int N)
   {
       mergeSort(A, 0, N - 1);

       return checkTriangle(A, N, 0);
   }

   int main()
   {
        int N;
        int i;

       printf("Fully Recursive Triangle Algorithm\n");

      printf("Enter number of inputs: ");
      scanf("%d", &N);

      int A[N];

     for(i = 0; i < N; i++)
     {
          printf("Enter side %d: ", i + 1);
          scanf("%d", &A[i]);
   }

   int result = triangleRecursive(A, N);

   if(result == 1)
   {
        printf("\n(1) Triangle exists\n");
```

```
    }
    else
    {
        printf("\n(0) Triangle does not exist\n");
    }
     return 0;
}
```

Analysis & Complexity:

Merge Sort Complexity: O(n log n)

Recursive Calls Complexity: O(n)

Final Time Complexity: O(n log n)

Space Complexity: O(n)

Comparison Between the 2 algorithms:

Feature	Non-Recursive	Recursive
Technique Used	Uses loops (for loop) to iterate through the array.	Uses Recursive Merge Sort and recursive function calls.
Working Method	The algorithm checks each triplet directly using iteration.	The algorithm recursively sorts the array using Merge Sort, then recursively checks each triplet.
Time Complexity	$O(n^2)$ because Bubble Sort is used.	O(n log n) because Recursive Merge Sort is used.
Space Complexity	O(1) because it uses only a few variables.	O(n) because Merge Sort uses additional arrays and recursive calls.
Memory Usage	Very low memory usage.	Higher memory usage because Merge Sort requires additional arrays and recursive calls.
Speed in Practice	Faster in real execution	Faster for large arrays because Merge Sort is more efficient.
Best Use Case	Better for real applications and performance-critical systems.	Better for efficient recursive processing and large datasets.